OGUNSOLA TOLUWASE PAUL

Backend & Distributed Systems Engineer | Node.js | TypeScript | Python

Oyo State, Nigeria | Open to Remote Roles & Contract Work

oluwaseyiogunsola90@gmail.com | +234 806 700 4668

GitHub | LinkedIn | Portfolio

PROFESSIONAL SUMMARY

Backend Software Engineer with **3** years of experience architecting distributed systems, real-time backends, and resilient AI infrastructure using TypeScript, Node.js, Python, PostgreSQL, and Redis. Specializing in high-concurrency correctness—from append-only financial ledgers and idempotent workflows to multi-provider LLM routing. Built agent-broker, an open-source TypeScript library leveraging atomic Redis Lua scripts to eliminate budget races, unbounded recursive delegation, and correlated retry storms across multi-process agent architectures, and continue extending it with new coordination capabilities.

Engineering Focus: Distributed State Coordination · High Concurrency & Consistency · Resilient AI Infrastructure · Idempotent Workflows

TECHNICAL SKILLS

Languages: TypeScript, JavaScript, Python, SQL
Backend & Real-Time: Node.js, Express.js, FastAPI, Flask, REST APIs, WebSockets, Socket.IO, Gunicorn, APScheduler
Databases & Messaging: PostgreSQL (RPCs, Row Locks, Indexes), Redis (Lua, Pipelines), BullMQ, RQ, Supabase, SQLAlchemy, SQLite
Testing & Infrastructure: Docker, Docker Compose, CI/CD (GitHub Actions), Sentry, Jest, FCM, Pytest, Vitest, Vercel, Railway, AWS, Cloudinary
Frontend Exposure: React.js

PROJECTS

agent-broker — Distributed Concurrency & Resilience Library | GitHub Repository

Creator & Systems Engineer | Sep 2026 – Present

TypeScript library coordinating cross-process execution, delegation limits, and cost reconciliation across independent Node.js processes via Redis. Core library complete; actively extending with new capabilities.

- **Atomic Admission Engine:** Fused budget reservations, concurrency caps, and circuit-breaker checks into a single atomic **7-key** Redis Lua script (EVAL), eliminating check-then-act race conditions across uncoordinated processes. [Admission Engine Code]
- **Delegation Tree Governance:** Enforced immutable, broker-derived agent hierarchies via an atomic REGISTER_CHILD Lua script that calculates depth and rootId in Redis, eliminating depth-spoofing and unbounded recursive delegation loops.
- **Two-Phase Cost Reconciliation:** Engineered a 2-phase API (requestPermission/reportOutcome) that reserves estimated costs pre-call and reconciles actual usage post-call via a unified resolution script (resolveReservation).
- **Sliding-Window Circuit Breaker:** Built a Redis Sorted Set failure tracker (ZREMRANGEBYSCORE) with dual-threshold backpressure and atomic probeRate draws inside the admission Lua script to prevent caller probe stampedes on recovering targets.
- **Multi-Process Test Harness:** Authored a **3-layer** test suite using a custom child_process.fork() IPC harness to verify atomic coordination under genuine OS process contention (e.g., **20 processes racing for 10 budget slots**).

StudyHub — AI & Real-Time Infrastructure | GitHub Repository | Live Demo

Creator & Backend Engineer | Jan 2024 – Present

Peer academic collaboration platform featuring a deployed multi-provider LLM failover engine and Redis-coordinated real-time services.

- **Database Schema & Indexing:** Engineered a **55-model** SQLAlchemy ORM schema on PostgreSQL, deploying **42** partial, GIN, and functional unique indexes (LEAST/GREATEST) to enforce relational invariants and eliminate bidirectional write race conditions.
- **Real-Time Presence & Pipelines:** Architected multi-device WebSocket presence using TTL'd per-socket keys cross-checked against untimed Redis user index sets (self-healing on read) to drive monotonic 3-state message delivery status (sent → delivered → read). [WebSocket Code]
- **Multi-Provider AI Failover Engine:** Architected a **6-provider** LLM failover engine (**14 models**) featuring Redis-cached dynamic model discovery, automated response sanitization, and mid-stream SSE provider switching with deterministic fallback. [AI Provider Code]
- **Fault Taxonomy & Circuit Breaking:** Classified LLM errors across 4 fault domains (KEY_FAULT, PROVIDER_TRANSIENT, BAD_MODEL, NON_RETRYABLE), isolating 5xx network outages from key cooldowns, enforcing **30-minute** provider blacklists, and gating state behind a fail-open Redis kill switch.
- **Distributed Locking & Background Queues:** Engineered a fail-closed Redis distributed lock (SET NX EX + token-verified Lua release) across 5 APScheduler jobs, pairing 2 durable RQ queues with an **8-worker** ThreadPoolExecutor for non-blocking dispatches.
- **Concurrency & Token Security:** Implemented SHA-256 refresh token rotation with family_id replay detection and a **10s** multi-tab grace window, pairing single-use preview-safe tokens with Google OAuth sub-claim binding to eliminate account hijack vectors.

Kith — Financial Coordination System | GitHub Repository | Live Demo

Creator & Backend Engineer | Mar 2026 – Sep 2026

Family financial coordination platform with an immutable ledger, automated recurring pools, and a 9-queue background job system.

- **Relational Schema & Constraints:** Designed a **22-table** PostgreSQL schema enforced by **59** foreign keys, **26** CHECK constraints, and **48** targeted indexes (**15** partial unique indexes enforcing single active target constraints per participant).
- **Concurrency-Safe Workflows (RPCs):** Encapsulated 7 multi-step write workflows inside atomic, row-locked PostgreSQL RPC functions, eliminating race conditions under concurrent requests and reducing write latency by **35%** under benchmarked stress tests.

- **Ledger Integrity & Idempotency:** Built an append-only immutable ledger with linked correction rows, client-supplied X-IdempotencyKey headers, a 10-minute duplicate-submission window heuristic, and an audit trail covering 30 action types across core financial API surfaces.
- **Transactional Outbox Architecture:** Engineered a transactional outbox pipeline for Firebase push and Resend email delivery, persisting delivery states in PostgreSQL with a 5-minute BullMQ recovery sweep for stranded jobs, push-token rotation checks, and dedup-keyed retries.
- **Isolated Queue Infrastructure:** Decoupled high-priority ledger reconciliation from scheduled payouts and low-priority notifications across a **9-queue** BullMQ architecture to eliminate head-of-line blocking, reducing API response times by **45%** under simulated load.
- **Distributed Authorization & Multi-Tier Rate Limiting:** Built a 30s Redis authorization cache with event-driven invalidation (**~40%** DB query reduction), paired with 6 Redis rate limiters split by IP (pre-auth enumeration defense) and User ID (post-auth shared-NAT protection).
- **Integration Testing & CI/CD:** Authored automated integration test suites across 3 Jest projects running against a 3-container PostgreSQL/PostgREST/Redis stack in GitHub Actions CI, achieving **90%+** critical-path coverage.

FounderSales — AI Sales Coaching Platform | GitHub Repository

Creator & Backend Engineer | *Feb 2025 – Aug 2026*

AI sales-coaching platform with cost-gated LLM pipelines, distributed concurrency guards, and request-level observability.

- **LLM Pipeline & Cost Optimization:** Built a pre-call cost gate and consolidated a 4-call simulation pipeline into 1 prompt returning 6 structured output dimensions, cutting request latency from **3.2s to 800ms** and LLM costs by **~60%**.
- **Distributed State & Concurrency Guards:** Engineered a Redis Sorted Set slot registry (maxConcurrent 15, 150ms stagger, 90s stale-slot sweep) to eliminate distributed deadlocks. [Concurrency Guard Code]
- **Tiered Rate Limiting & Queue Workers:** Protected auth, LLM execution, and analytics tiers using **19** namespaced Redis rate limiters alongside a 3-queue BullMQ worker architecture running **21** cron schedules with 5 distinct idempotency strategies.
- **Schema Design & Modular API:** Designed a **48-table** PostgreSQL schema with **98** foreign keys, pushing race-prone writes into 10 stored procedures across 20 modular API routers with fine-grained RBAC across 4 roles.
- **Context Caching:** Cut 2–3 DB round-trips per request via 30s Redis workspace-context caching (ws:ctx) with instant event-driven cache invalidation.
- **Observability & Test Rigor:** Implemented X-Trace-Id request-tracing middleware with non-blocking Sentry error logging and Vitest unit test suites in CI.

EDUCATION

Federal University of Technology, Akure (FUTA) — Ondo State, Nigeria
Bachelor of Science (B.Sc.) in Computer Science — Expected Graduation: 2028